

When Working JavaScript Breaks

Browser Security Policies in Practice

Elena Darevskaya

Senior Platform Engineer, SAP

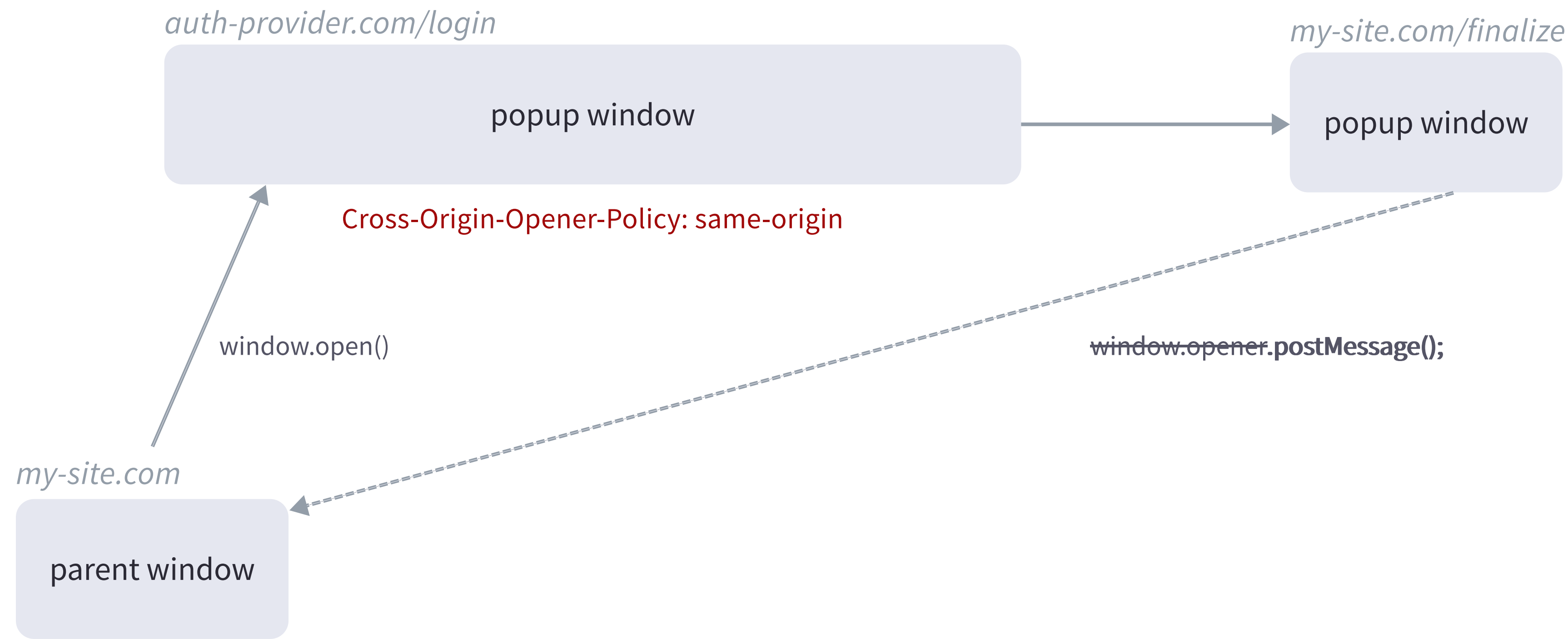
How do we make the policy secure?

How do we make the code compatible with it?

Why does this matter if we can control the environment?

- Earlier is cheaper
- Not every flow gets tested
- AI-generated code needs constraints

Authentication in popup



What assumption did we make during the design?

The popup will always have access to `window.opener`

COOP header breaks this assumption

Questions to ask during design

- Does it depend on a relationship between windows, popups or origins?
- Can a security header block what this code is trying to do?
- Does this code require a secure context?
- Can Permissions Policy restrict it?

Make these questions part of the workflow

Use MDN MCP in AI-driven development and review

Define how browser APIs may be used

- Fully block some browser APIs
- Allow using some APIs only in specific ways

Block Eval

eslint.config.json

```
{
  // eval("x = 1") or window.eval("x = 1")
  "no-eval": "error",

  // new Function("a", "return a")
  "no-new-func": "error",

  // setTimeout("doThing()", 100)
  "no-implied-eval": "error"
}
```

Block or restrict innerHtml

```
element.innerHTML = html  
element.outerHTML = html  
element.insertAdjacentHTML(...)
```

Allow using some APIs only in specific ways

eslint.config.js

```
{
  'no-restricted-syntax': [
    'error',
    {
      selector:

      "AssignmentExpression[left.property.name=/^(innerHTML|outerHTML)$/]
      :not([right.callee.name='sanitizeHtml'])",
      message: sinkMessage,
    }
  ]
}
```

You can also use existing plugins:

- eslint-plugin-no-unsanitized
- eslint-plugin-browser-security

Forcing architecture with static checks: Worker

```
const url = URL.createObjectURL(  
  new Blob([workerCode], { type: "text/javascript" })  
);  
  
new Worker(url);
```

Forcing architecture with static checks: Worker

worker-factory.js

- Only place allowed to call new Worker()
- Owns worker URL creation
- exposes createWorker()

Other files

Only allowed to use createWorker() from worker-factory.js

Are we now safe from eval?

eslint.config.json

```
{  
  // eval("x = 1") or window.eval("x = 1")  
  "no-eval": "error",  
  
  // new Function("a", "return a")  
  "no-new-func": "error",  
  
  // setTimeout("doThing()", 100)  
  "no-implied-eval": "error"  
}
```

Bundlers can introduce security risks

What can appear in the build

- eval
- Inline styles
- Dynamic scripts
- blob: / data: resources

How to catch it

- Lint the config itself
- Scan the build artifact for possible issues introduced by bundler

Testing with Browser Security Policies in mind

- Run the same tests with different sets of security headers
- Collect security violations during the test run
- Make sure the things we expect to be blocked are actually blocked

Testing with your security header

playwright.config.json

```
{
  projects: [
    {
      name: 'permissive',
      use: {
        csp: "script-src 'self'; worker-src 'self' blob:"
      }
    },
    {
      name: 'strict',
      use: {
        csp: "script-src 'self'"
      }
    }
  ]
}
```

Testing with your security header

inside beforeEach or fixture

```
const { csp } = testInfo.project.use;

await page.route('**/*.html', async (route) => {
  const response = await route.fetch();
  await route.fulfill({
    response,
    headers: { ...response.headers(), 'content-security-policy': csp },
  });
});
```

Catch the violations during tests

inside beforeEach or fixture

```
await page.addInitScript(() => {
  window.cspReports = [];
  new ReportingObserver(
    (reports) => {
      for (const { body } of reports)
        window.cspReports.push({
          directive: body.effectiveDirective,
          blockedUrl: body.blockedURL,
          ...
        });
    },
    { types: ['csp-violation'], buffered: true },
  ).observe();
});
```

Production monitoring: Report-Only headers

Use Report-Only security headers to monitor policy violations on production:

- CSP
- COOP
- COEP
- Permission Policy
- Browser API deprecations

Production monitoring

Headers

```
Reporting-Endpoints: default="https://example.com/reports"
```

```
Content-Security-Policy-Report-Only:  
  default-src 'self'; report-to default
```

Production monitoring: Report format

```
[
  {
    "type": "csp-violation",
    "url": "https://example.com/orders/93847?mode=edit",
    "user_agent": "Mozilla/5.0 ...",
    "body": {
      "blockedURL": "inline",
      "columnNumber": 5,
      "disposition": "report",
      "documentURL": "https://example.com/orders/93847?mode=edit",
      "effectiveDirective": "script-src-elem",
      "lineNumber": 18,
      "originalPolicy": "default-src 'self'; report-to default",
      "referrer": "https://example.com/account",
      "sample": "",
      "sourceFile": "https://example.com/orders/93847?mode=edit",
      "statusCode": 200
    }
  }
]
```

Production monitoring without headers

- **securitypolicyviolation**
 - CSP-specific
 - broadly supported
- **ReportingObserver**
 - supports multiple report types
 - can capture reports created before the observer starts
 - Newer API, browser support may differ per report

Reporting through headers is a recommended way, but third-party scripts don't have this option

Make browser security constraints visible at every stage

Design

Ask what assumptions the browser API makes

Develop

Block unsafe browser APIs and enforce architecture with static analysis tools

Build

Check bundler configurations and scan build artifacts

Test

Run with different policies and track the violations

Monitor

Use Report-only headers or ReportingObserver to track violations

Thank you!



<https://github.com/darevskaya/when-working-javascript-breaks>



<https://www.linkedin.com/in/edarevskaya/>

MDN MCP:

<https://github.com/mdn/mcp>

Useful eslint plugins:

- [eslint-plugin-browser-security](#)
- [eslint-plugin-no-unsanitized](#)